

RAG架构与原理详解

主讲人：龙鹏（笔名言有三）

目录



- 背景与定义
- 数据切分详解
- 嵌入模型详解
- 向量库与检索详解

背景与定义

大模型的局限性



- LLM包含的是参数化知识，能够对一般的指令作出响应，但存在知识截止与幻觉问题。

谁是2024年巴黎奥运会女子
单打网球冠军？



不知道

龙鹏的高中数学老师叫什么



不知道

两个常见的解决方案： **(1) 模型微调； (2) 本地知识库**

- 微调是进一步提升大模型能力、开发行业大模型的关键技术

01

提高领域适应性

微调可以帮助模型更好地适应特定需求，如对敏感数据（如医疗记录）的处理。

02

增强意图理解能力

微调可以进一步提升模型的用户意图理解能力，进一步提升模型的zero-shot能力。

03

减少计算资源

微调可以提高部署效率、减少计算资源需求。

- 微调并不适用于所有场景

数据准备成本高

需要收集、整理和标注大量特定领域的数据，这是一个耗时费力的过程。标注数据的质量直接影响微调的效果，需要确保数据标注的准确性和一致性。

时效性问题

对知识更新频繁的领域，微调后的模型可能很快会过时，需要不断重新训练。

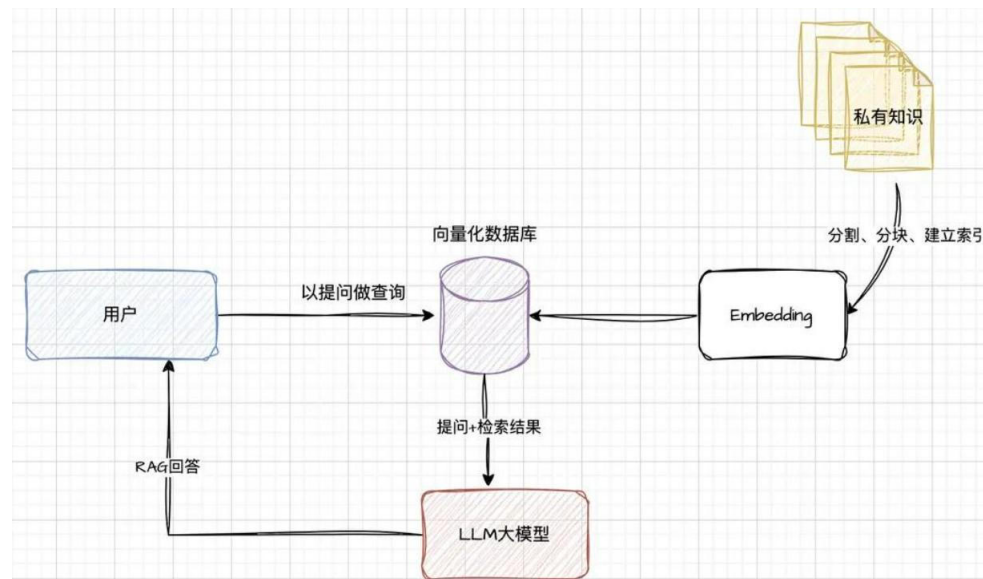
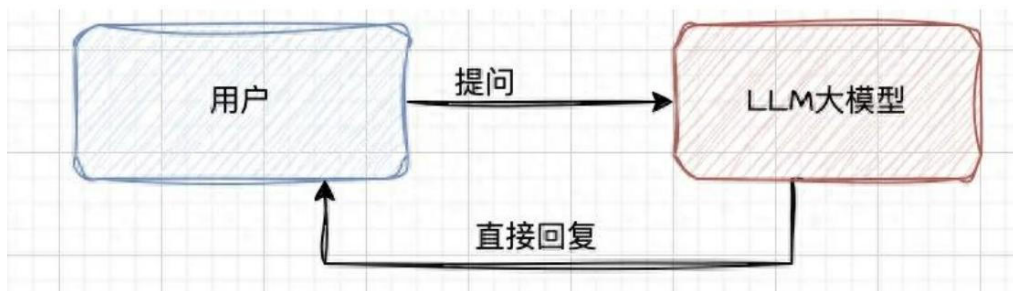
技术门槛

需要有算法经验的专业技术人员来完成，选择合适的微调算法和超参数，避免过拟合或欠拟合问题。

- 采用检索增强生成RAG(Retrieval-Augmented Generation)的技术方案

核心思想：在生成答案前，先从外部知识库中检索相关信息，再将检索结果与用户输入结合，指导生成模型输出更可靠的回答。

具体地说，就是利用已有的文档等资料生成向量知识库，在提问的时候结合从库中检索到的内容一起给大模型，让其回答结果更加准确，它**结合了信息检索和大模型技术，旨在提升模型生成内容的准确性和相关性**



- RAG 几乎可以被任何 LLM 用于连接任意外部资源，因此作者把 RAG 称为 “通用的微调秘方”

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

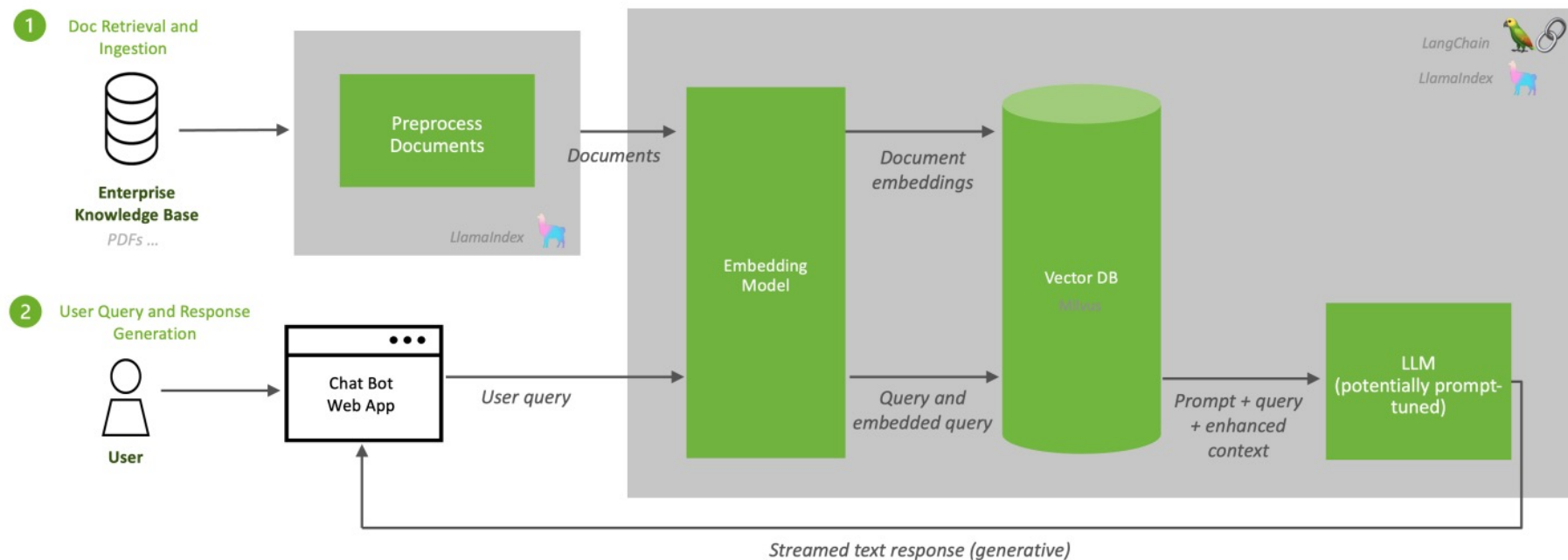
Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; ^{*}New York University;
plewis@fb.com

- 检索增强生成成为模型提供了可以引用的来源，就像研究论文中的脚注一样。这样用户就可以对任何说法进行核实，从而建立起信任。还能帮助模型消除用户查询中的歧义，降低模型做出错误猜测的可能性，该现象有时被称为 “幻觉”。
- 开发者只需几行代码就能实现这一流程。这使得该方法比使用额外的数据集来重新训练模型更快、成本更低，而且还能让用户随时更新新的来源。
- 几乎所有企业都可以将其技术手册、视频或日志转化为 “知识库” 资源，以此增强 LLM。

RAG的完整流程

• 完整流程



- ① 离线创建知识库，将企业知识如各类文档经过预处理后（主要是数据进行分段以及过滤）发送给嵌入模型，处理后得到向量统一存入向量库（即知识库）。
- ② 当用户向 LLM 提问时，模型会将用户提问（即查询）发送给嵌入模型，其将查询转换成向量。
- ③ 嵌入向量与知识库里的向量进行比较。当发现存在一个或多个匹配项时，将对应人类可读的原字符串（即数据片段）发送回 LLM。
- ④ 最后，LLM 将检索到的结果和查询相结合，发送给大语言模型进行整理，得到最终的答案并返回给用户，其中可能会引用嵌入模型找到的来源、即检索到的数据片段。
- ⑤ 在后台，嵌入模型可以不断创建并更新知识（向量数据库）。

RAG的特点



- RAG技术特点

保护数据隐私

由于日常的业务知识是保存到本地的，所以减少信息泄露的风险

减少模型幻觉

由于提问结合了业务知识，所以减少了模型的幻觉，即减少了模型的胡说八道

实时知识补充

模型的回复结合了业务知识和实时知识，所以实时性可以更好

无需重新训练

不用重新训练模型或微调模型，降低了成本

RAG的典型场景

- 检索文档资料，生成回答



文档型问答

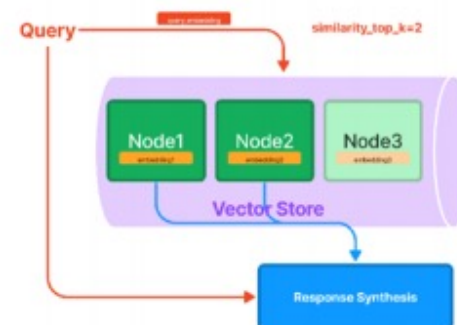


客服机器人

客服中心辅助

教育答疑

教育测试



数据分段详解

- 数据分段是将原始的长文档（如PDF、网页、书籍）按一定规则切分成多个较小的、语义相对独立的文本片段（称为“块”， chunk）。每个块后续会被转换成向量。

假设你有一本《Python入门教程》的电子书，总共 300 页，里面包含从“安装Python”到“面向对象编程”的全部内容。你问 RAG 系统：“如何定义一个类？”

不分段处理

- 系统必须把整本书 300 页的内容全部塞给大模型，但大模型有上下文长度限制（如 8k token），很可能放不下。
- 即使勉强放下，检索到的相关片段也被淹没在大量无关文字中，大模型难以精确回答。

分段处理

块ID	内容来源	片段内容（示例）
1	第1章 “安装与环境配置”	“Python可在官网下载，安装后需配置环境变量...”
2	第3章 “变量与数据类型”	“Python中变量无需声明类型，如 x=5...”
3	第5章 “函数定义”	“使用 def 关键字定义函数，例如 def add(a,b):...”
4	第7章 “类和对象”	“通过 class 关键字定义类，__init__ 是构造方法...”

- RAG 系统将问题转换成向量。
- 在向量数据库中搜索最相似的块。
- 找到块4（第7章内容）并返回。
- 大模型只看这一小段（几百个字）就能精准回答：“使用 class 关键字...”。

- 数据分段的主要作用

作用	说明
突破上下文长度限制	大模型无法一次性处理几十万字的文档，分段后可只取最相关的几千字。
提高检索精准度	小块语义集中，向量相似度计算更准，避免整篇文章中的噪声干扰。
降低推理成本与延迟	送入大模型的 token 数显著减少，处理更快、更便宜。
支持更细粒度的引用	可以告诉用户答案来自文档的哪个具体段落或页码。

- 数据分段的核心是在 “保持语义完整性” 和 “便于精准检索” 之间找到最佳平衡

设计维度	核心目标	权衡与挑战
语义完整性	确保切分后的“块”是独立的、意义完整的语义单元。	块过大可能包含多个主题，引入噪声；块过小则可能割裂关键信息，导致信息不完整。
检索效率与精度	切分的块应作为检索的基本单位，直接匹配用户问题。	块是检索的粒度，需要在“召回相关块”和“避免无关信息”之间取得平衡。
上下文边界管理	块的大小决定了其嵌入向量的表征质量和LLM处理时的上下文成本。	块过长会稀释关键信息，增加计算负担；块过短则可能缺乏足够的上下文进行准确理解。

基础切分方法



- 基础切分构件通常直接由各类RAG框架（如LangChain）提供

方法	具体操作	适用场景	注意事项
固定大小分块	按字符数或词数进行简单切割。	对结构、语义要求不高的通用文档（如日志）。	可能粗暴地切断句子或段落，破坏语义。
按分隔符分块	利用段落、标题、句号等自然标记进行分割。	结构清晰的文档（如Markdown、代码）。	分隔符的选择直接影响块的质量。
滑动窗口分块	在固定大小的基础上设置重叠区，避免边界信息丢失。	绝大多数文本的通用改进方案。	重叠区大小需要调优，会增加索引总量。
多层分块	先尝试用大分隔符（如\n\n）分块，如果块太大再用小分隔符（如\n）递归切分。	处理结构复杂的嵌套文档（如PDF）。	能在不同层级上保持结构，是常用且稳健的策略。

高级语义切分方法



- 旨在实现更智能、更贴合内容结构的切分

方法	核心思想	关键技术/工具	优点与挑战
基于模型/语义的分块	利用NLP模型理解文本，在语义边界（如主旨变化处）进行切分。	使用嵌入模型计算句子向量，检测向量距离的突变点；或使用文本分割模型。	能最大程度保持语义连贯性，是当前前沿方向。但对算力有要求，且依赖模型质量。
基于版面/视觉的分块	对于扫描PDF、图像等，先识别版面元素（标题、段落、图表），再按逻辑单元切分。	OCR技术（如Tesseract）、版面分析模型（如LayoutParser）。	能还原非纯文本文档的原始结构。技术栈更复杂，涉及多模态处理。
多级索引与混合粒度	构建不同粒度的索引层次，例如同时存储“小节”块和其下的“段落”块。	在检索时可根据问题灵活返回不同粒度的信息，或进行融合。	兼顾召回精度和上下文丰富度，系统设计和检索逻辑更复杂。

嵌入模型详解

自然语言的数字化表示

- 向模型直接输入字符串显然是不可行的，不便于模型进行计算和文本之间的比较。模型的输入必须是数值型向量。

今天是个大晴天

Compute

	0	1	2	3	4	5	6	7	8	9	10	
0	0.175	0.065	-0.214	-0.041	0.393	-0.195	-0.149	-0.414	0.190	0.296	0.556	0.
1	0.198	-0.213	-0.042	0.216	0.173	-0.340	-0.168	-0.515	0.348	0.212	0.673	-0.
2	0.368	-0.164	-0.121	-0.080	0.265	-0.427	0.021	-0.579	0.282	0.293	0.684	-0.
3	0.464	0.026	-0.079	0.110	0.216	-0.271	-0.096	-0.479	0.458	0.189	0.599	-0.
4	0.521	0.053	-0.063	0.144	0.181	-0.267	-0.077	-0.565	0.423	0.259	0.667	-0.
5	0.342	-0.069	-0.046	0.245	0.215	-0.185	-0.056	-0.612	0.210	0.099	0.815	-0.
6	-0.066	-0.282	-0.123	0.196	0.331	-0.188	0.050	-0.729	0.390	0.076	0.604	-0.
7	0.312	-0.146	-0.169	-0.023	0.255	-0.300	-0.013	-0.580	0.264	0.246	0.618	-0.
8	0.175	0.065	-0.214	-0.041	0.393	-0.195	-0.149	-0.414	0.190	0.296	0.556	0.

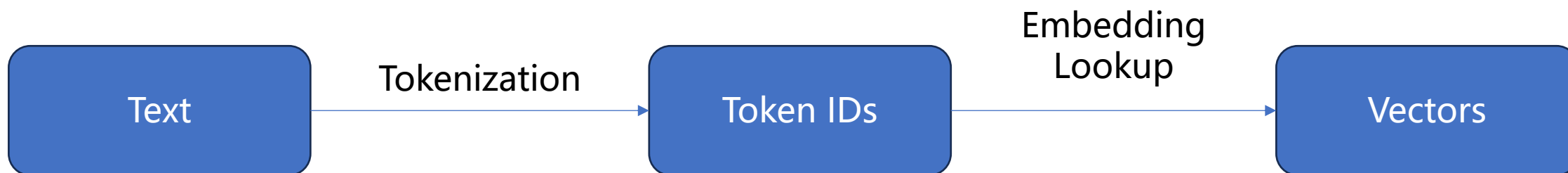
两个基本任务



文本字符串经过计算后变为浮点型向量

现代词向量获取方法

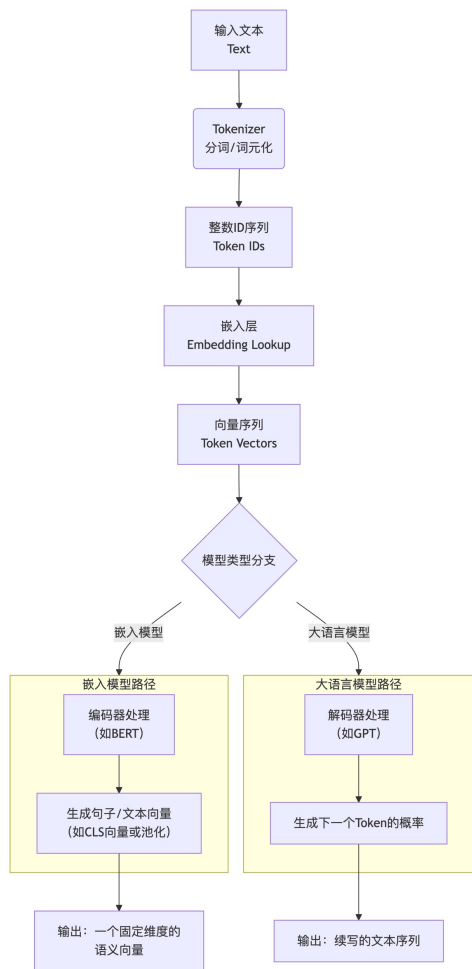
- 字符串被拆分为模型能理解的离散基本单元 (Token)，然后每个Token被映射为一个数字ID，最终转换为向量。



使用同类的 Tokenizer (如BPE、WordPiece)，将文本转换为离散的整数ID，再通过模型内部的 Embedding Layer (一个巨大的查找表) 将每个ID转换为一个稠密的词向量。 **Embedding Layer**用于学习词向量。

两类模型比较

- 嵌入模型与大语言模型的区别



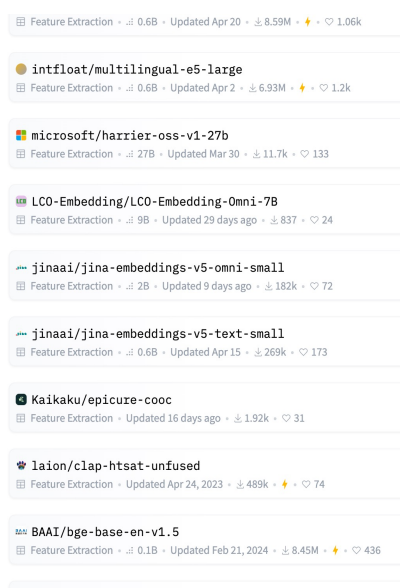
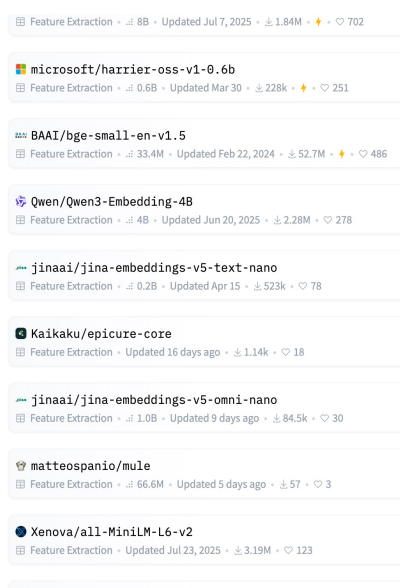
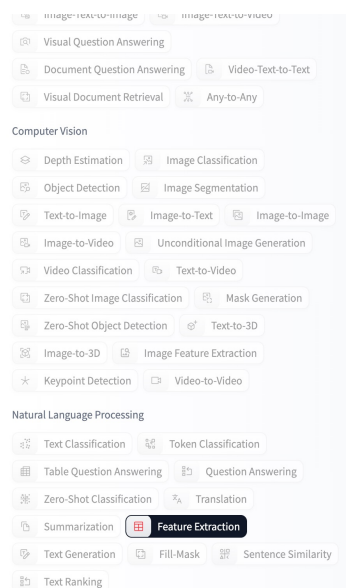
在获得**向量序列**之后，两类模型的目标和架构导致了路径的分叉：

- 嵌入模型**的目标是生成一个**代表整段文本语义的、固定长度的向量**。因此，它通常使用**编码器架构**（如BERT），通过“CLS”位或对所有词向量进行池化（平均、最大等）来“综合”整个序列的信息，最终输出**一个向量**。
- 大语言模型**的目标是**生成文本**。因此，它通常使用**解码器架构**（如GPT），基于当前序列去计算下一个词元的概率分布，并进行自回归采样，最终输出**续写的文本序列**。

嵌入模型体验



- 去Huggingface或modelscope社区自然语言处理标签下选择模型



向量库与检索详解

不同检索方式



- 常用的检索策略

关键词检索

通过用户输入的信息文本在知识库全文匹配，适用于**用户知道确切的信息或术语的场景**。该方法所消耗的计算资源较低，适合在大量文档的知识库内快速检索。

向量/语义检索

借助嵌入模型，**即便知识库中没有出现查询中的确切词汇**，也能通过**计算向量距离的方式**返回相似内容。此外，当**需要处理多语言内容**时，语义检索能够捕捉不同语言之间的意义转换，提供更加准确的跨语言搜索结果。

混合检索

同时使用两种检索方式，并通过分配不同的语义和关键词权重来对结果进行平衡，选择最优的效果。

- 向量数据库是专为存储和检索高维向量数据设计的数据库系统，通过相似度计算实现高效查询，支撑RAG系统的语义搜索能力

技术演进需求

RAG系统依赖向量数据库快速匹配用户的查询。传统关键词检索难以处理语义关联，向量数据库通过嵌入技术将文本映射为向量空间，解决了复杂语义匹配的瓶颈问题。

核心功能与原理

向量数据库的核心功能包括向量索引构建、近似最近邻搜索和实时更新，确保大规模数据下的快速检索与准确度，从而提供大规模向量的存储、索引、查询等一体化服务。

应用场景

电商推荐系统：海量相似商品快速检索。

医疗影像分析：基于内容的相似病例检索

金融风控：比对用户行为特征向量，实时识别异常交易模式

向量数据库优势



- 检索效率、扩展能力、精度

检索效率

向量数据库在高维数据检索中显著优于传统数据库，支持毫秒级相似度匹配，适用于RAG系统的实时响应需求。

扩展能力

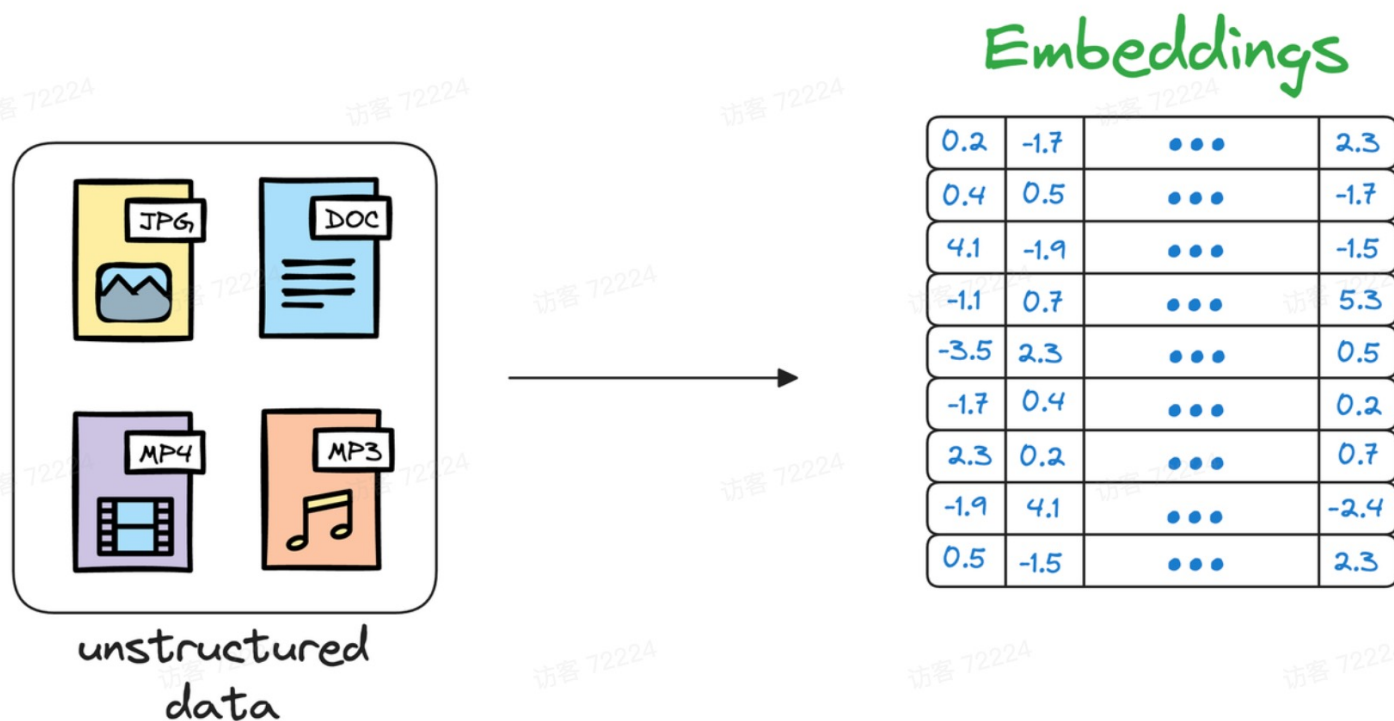
分布式向量数据库可横向扩展，轻松处理亿级数据规模，而传统方案在数据量激增时性能急剧下降。

精度和召回率

基于ANN算法的向量数据库在精度与召回率间实现平衡，相比精确检索牺牲少量精度换取百倍速度提升。

向量数据库计算原理

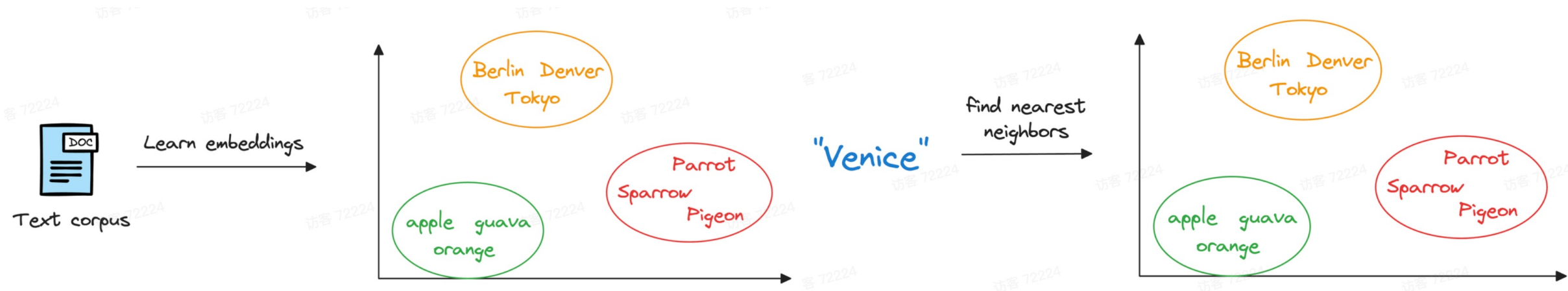
- 向量编码：将非结构化数据（文本图片等）转化为机器可计算的、蕴含语义的向量



关键技术：使用Embedding模型（如BERT、M3E、text-embedding-3-large等）生成高维稠密向量（如768维），用于捕获深层语义关联。通过微调和降维技术提升向量表征质量，解决传统关键词检索的语义鸿沟问题。

向量数据库计算原理

- 相似度计算与检索：在向量空间中，找出与查询向量“最相似”的数据向量



- 相似度度量：常用余弦相似度，计算向量间的夹角。
- 高效检索：为避免对海量向量进行穷举计算，采用近似最近邻搜索(ANN)算法（如HNSW、IVF-PQ）在精度和速度间平衡，实现毫秒级检索

向量数据库计算原理



- 近似最近邻搜索算法 (ANN、Approximate Nearest Neighbor)

如果采用精确搜索（暴力遍历所有向量），每次查询都要计算所有片段与问题的距离。当知识库有数千万条时，一次查询可能花费几秒钟甚至更久，用户体验完全不可接受。**ANN 让 RAG 在毫秒级完成检索，代价是可能遗漏极少数本来应该排在前面的片段**，但通常这种误差在可接受范围内。

倒排索引 + 聚类 (IVF、Inverted File)

原理：用聚类算法（如 K-means）把所有向量分成几百或几千个簇。查询时，先找到离问题向量最近的几个簇，然后只在这些簇内部搜索。

应用：广泛应用，比如 Faiss 库中的 IndexIVFFlat

层次可导航小世界图 (HNSW、Hierarchical Navigable Small World)

原理：把向量建成多层图结构。顶层节点稀疏，底层节点密集。查询时从顶层进入，沿着“邻居”关系快速跳跃到底层，找到最近的点。

应用：目前最流行、效果最好之一，许多向量数据库（如 Milvus、Qdrant）的默认索引方法。

当前主流向量数据库产品



• 一些国产向量数据库

产品名称与公司	核心定位	主要特点与适用场景	部署方式
Milvus 厂商：Zilliz	开源的高性能、可扩展向量数据库，专为海量向量检索而生。	特点： 云原生分布式架构，计算存储分离，支持 十亿级向量 。提供丰富的索引算法和搜索类型。 场景： 大模型知识库、AI语义搜索、推荐系统、多模态检索。	灵活：提供 Lite （Python库）、 Standalone （单机）、 Distributed （分布式集群）及全托管云服务 Zilliz Cloud 。
VectorDB 厂商：百度智能云	百度自研的 全托管专业向量数据库 ，基于“莫愁”内核。	特点： 开箱即用 ，支持百亿向量规模。集成混合检索、内置多租户与异地多活等企业级功能。 场景： 大模型知识库、智能问答、推荐系统。	全托管云服务，提供多种实例规格。
DingoDB 厂商：九章云极 DataCanvas	开源多模态向量数据库 ，强调 结构化与非结构化数据融合分析 。	特点： 统一存储标量与向量数据，支持 一体化SQL引擎 进行联合查询。 场景： 需要同时进行数据分析与语义搜索的复杂场景，如企业知识库、智能决策。	支持开源部署。
TiDB 厂商：PingCAP	开源分布式关系型数据库 ，正在集成 向量搜索 功能。	特点： 作为 HTAP数据库 ，其核心是处理关系型数据。其向量搜索旨在 简化技术栈 ，实现关系数据、向量、图数据的统一处理。 场景： 希望用单一数据库同时支撑业务系统和AI（GraphRAG）应用的场景，尤其适合已使用TiDB的团队。	开源软件，可自行部署；也提供TiDB Cloud托管服务。
openGauss 厂商：华为	开源企业级关系型数据库 ，可通过扩展支持向量检索。	特点： 内核面向事务与分析优化。其向量能力通常通过插件或衍生版本（如 Vastbase ）实现，与生态紧密集成。 场景： 在华为或openGauss生态内，需要将向量搜索与核心业务数据结合的场景。	开源软件，可自行部署或使用商业发行版。
AnalyticDB 厂商：阿里云	阿里自研的 云原生数据仓库 ，支持向量检索。	特点： PB级实时分析能力 强大。向量检索作为其 多模查询 的一部分，适合在数据仓库内直接对分析结果进行语义搜索。 场景： 需要 同时进行大数据分析、BI报表和向量相似性搜索 的复杂混合负载场景。	全托管云服务。

- 追求专精向量检索与极致规模：首选 Milvus。
- 需要全托管服务，快速集成：选择 百度VectorDB 或 阿里云AnalyticDB（如果同时有强数据分析需求）。
- 强调多模态数据融合与联合分析：关注 DingoDB。
- 希望统一技术栈，用一款数据库覆盖业务与AI：评估 TiDB（关注其向量功能进展）或基于 openGauss 的衍生方案。

扫一扫，关注有三AI公众号



三人行，必有AI